LAB REPORT – LAB 3: Motors & MOSFETs

Group Members: Milan Khadka and Udit Bhattarai
Group Members: Lotus Nyaupane and Dipen Gaihre

Course: Controllers and Electronics

Date: 28.11.2025

Note: This experiment was completed by a group of two students, and we used AI as writing assistants to improve the clarity of the report and to generate code in lab while performing different tasks.

Additionally, We member Lotus and Dipen had done this experiment with the another group of Milan and Udit due to lack of computer during the lab Session.

1. Objectives:

The goal of this lab was to learn how to control a DC motor using a Raspberry Pi and a MOSFET as a switch. Since the Pi cannot power a motor directly, we had to use an external 12V supply and control it safely through the MOSFET.

The key objectives for this were:

- Understand the operating principles of DC motors and MOSFET transistors.
- Build a pull-up resistor circuit for a button input.
- Control a motor through a MOSFET using Raspberry Pi GPIO.
- Implement a callback-based button input for more accurate and responsive motor control.
- Practice building safe motor-driving circuits including flyback diodes.

During the lab, we practiced building a pull-up resistor for a button, switching a motor on and off from Python code, and finally using callback functions for detecting button presses more accurately. The overall objective was to understand how to interface the Pi with external electronics safely and how MOSFETs work in real circuits.

2. Theory and Pre-Lab Work:

This section explains the theoretical background, as required in the "A good lab report – Hyvä laboratorioseloste" guidelines.

2.1 DC Motor Basics:

A DC motor converts electrical energy into mechanical rotation by using electromagnetic fields. When current flows through the motor windings, it creates magnetic forces that rotate the shaft. In this lab, a **12V brushed DC motor** was used and powered from an external power supply, since the Raspberry Pi cannot supply sufficient current or voltage. This is why we needed a separate power supply and a MOSFET to control when the motor receives current.

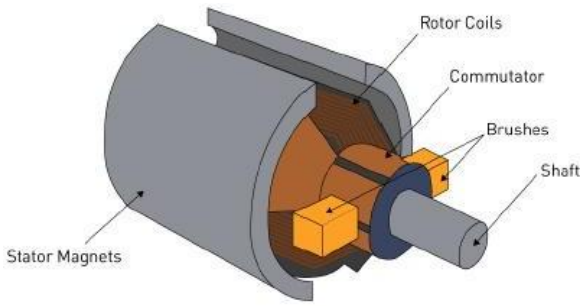

Figure 1: Brushed DC motor break

2.2 MOSFET Theory:

A MOSFET (Metal-Oxide-Semiconductor Field-Effect Transistor) acts as a voltage-controlled switch.

We used an **IRL540** N-channel logic-level MOSFET.

From the datasheet:

- Gate-source threshold voltage **VGS(th): 1.0–2.0V**
- Meaning: The MOSFET begins to turn on at around 1–2V gate voltage.
- This is why IRL series MOSFETs are logic-level, suitable for Raspberry Pi's 3.3V output.

Thus, 3.3V from Pi is enough to fully turn the MOSFET on.

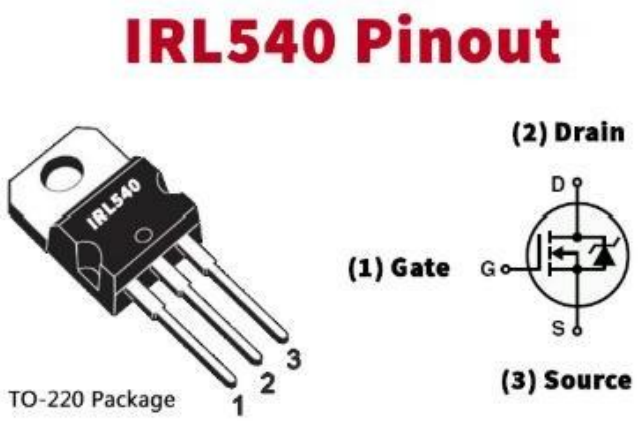

Figure 2: IRL540 N-channel logic-level MOSFET

2.3 Pull-Up Resistor:

A button input without a pull-up or pull-down resistor tends to "float", which means it produces random values. A pull-up resistor keeps the GPIO pin at a stable HIGH level when the button is not pressed.

In this lab we built a **10kΩ external pull-up resistor**, instead of using the Pi's internal one, to understand the concept better.

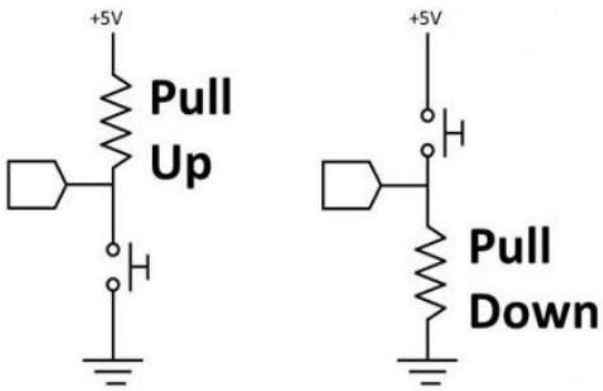

Figure 3: Pull-Up and Pull-Down Resistor

2.4 Flyback Diode:

When a motor stops, it produces a reverse voltage spike that can damage electronics. A diode placed across the motor provides a safe discharge path.

So, we used Flyback Diode in this lab.

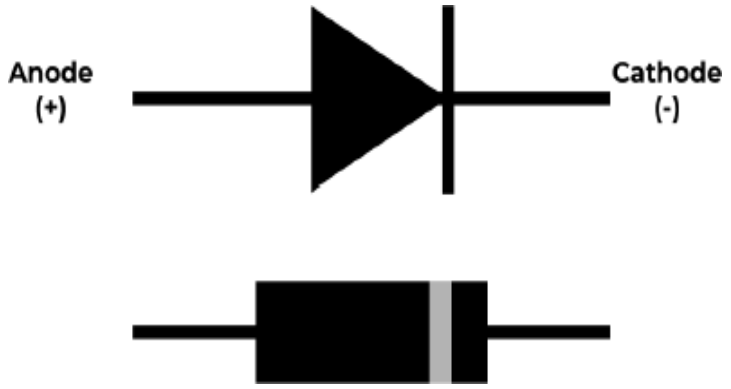

Figure 4: Diode graphic

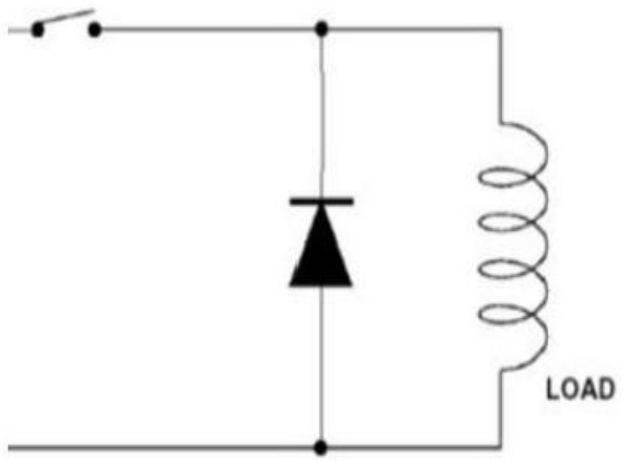

Figure 5: Flyback diode with a coil

3. Equipment, Materials andMethods:

3.1 Equipment & Components:

Device	RaspberryPi Zero 2W
Libraries	Pigpio time

Components	Amount	Datasheet
DC Motor 12V (FIT0492-B)	1	https://www.farnell.com/datasheets/3708795.pdf
External 12V power supply	1	
MOSFET (IRL540)	1	https://www.vishay.com/docs/91300/irl540.pdf
Resistor (10k Ohm)	2	
Diode	1	
Button	1	
40pin-connector	1	
Cables (M-M)	enough	
Breadbord	1	
USB micro - power cable	1	

Table 1: Equipment and components list

3.2 Wiring Setup:

We followed the wiring diagram from page 7 of the lab instructions, making sure not to accidentally connect the 12V supply directly to the Pi pins.

The wiring follows the schematic provided in the instructions (page 7) :

- Motor connected to 12V supply
- MOSFET source → ground
- MOSFET drain → motor
- MOSFET gate → Raspberry Pi GPIO pin via software control
- Flyback diode across motor terminals
- Button → GPIO input with external 10k pull-up resistor to 3.3V

Below are photos of our final wiring. The full circuit includes the MOSFET, button, pull-up resistor, flyback diode, motor, and Pi connection.

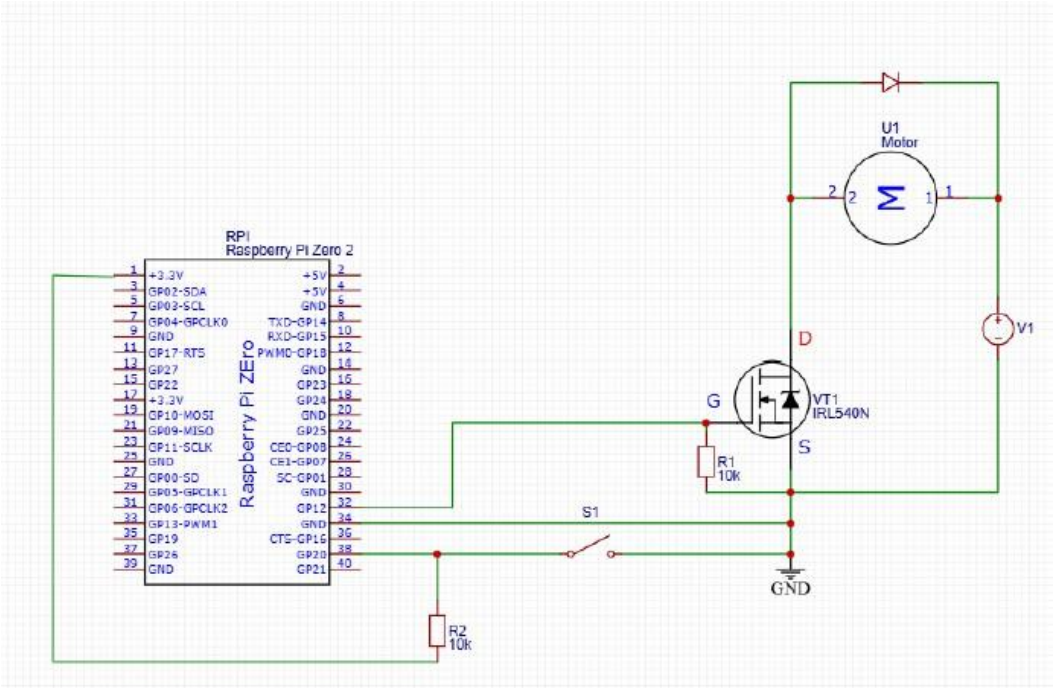

Figure 6: Full Circuit layout

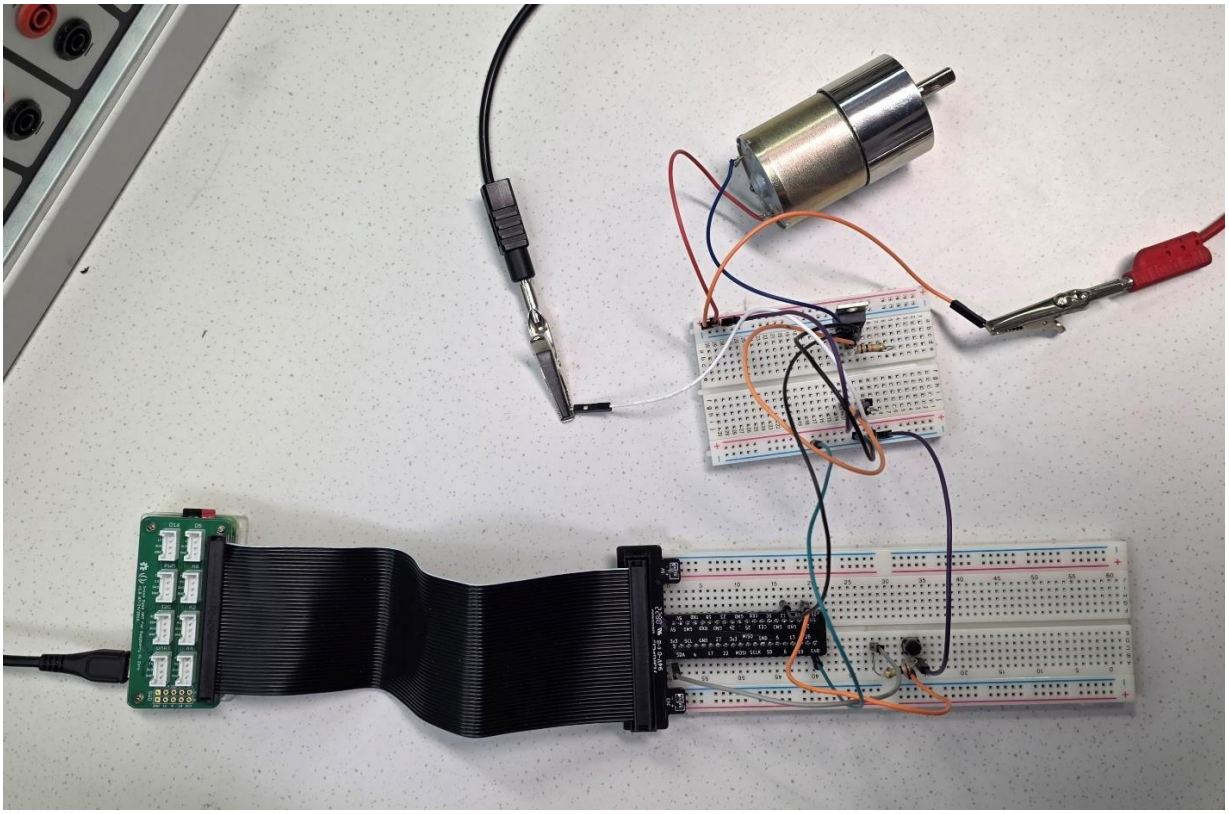

Figure 7: Photo taken in lab

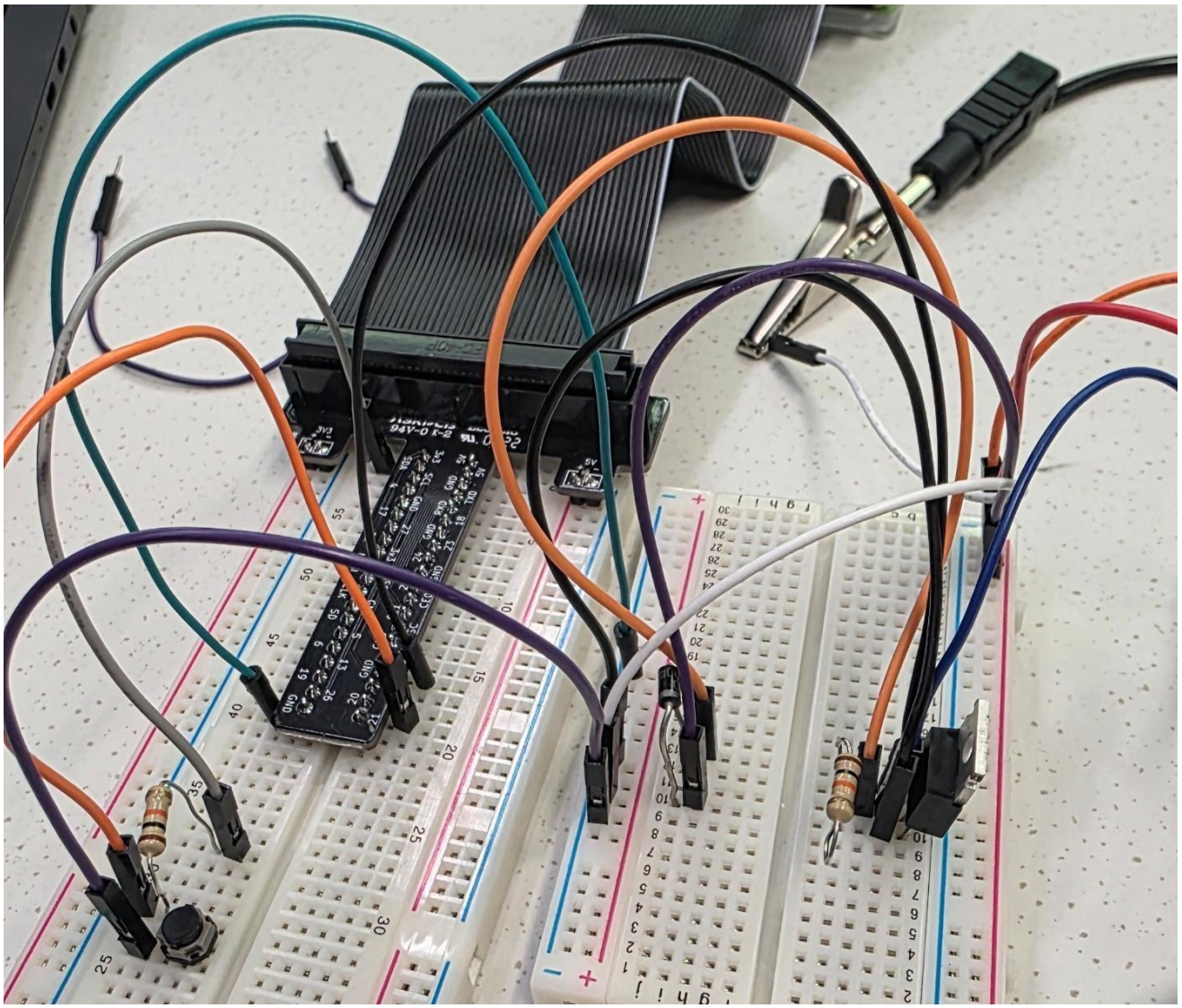

Figure 8: Photo taken in lab

4. Task Perform in Lab 3:

In this lab 3 we successfully performed the 4 tasks. The screenshot of the code we used in different tasks are listed below with their explanation.

Task 1: MOSFET Datasheet Study

(1) From the IRL540 datasheet, the threshold voltage VGS(th) is between **1.0 and 2.0 volts**. It is on page 2 of the Datasheet. I also include the screenshot of it below.

SPECIFICATIONS (T_J = 25 °C, unless otherwise noted)							
PARAMETER	SYMBOL	TEST CONDITIONS		MIN.	TYP.	MAX.	UNIT
Static							
Drain-source breakdown voltage	V_{DS}	V_{GS} = 0 V, I_D = 250 µA		100	-	-	V
V_{DS} temperature coefficient	$\Delta V_{DS}/T_J$	Reference to 25 °C, I_D = 1 mA		-	0.12	-	V/°C
Gate-source threshold voltage	$V_{GS(th)}$	V_{DS} = V_{GS}, I_D = 250 µA		1.0	-	2.0	V
Gate-source leakage	I_{GSS}	V_{GS} = ± 10 V		-	-	± 100	nA
Zero gate voltage drain current	I_{DSS}	V_{DS} = 100 V, V_{GS} = 0 V		-	-	25	µA
		V_{DS} = 80 V, V_{GS} = 0 V, T_J = 150 °C		-	-	250	
Drain-source on-state resistance	$R_{DS(on)}$	V_{GS} = 5.0 V	I_D = 17 A[b]	-	-	0.077	Ω
		V_{GS} = 4.0 V	I_D = 14 A[b]	-	-	0.11	
Forward transconductance	g_{fs}	V_{DS} = 50 V, I_D = 17 A		12	-	-	S

Table: threshold Voltage

(2) This confirms that the MOSFET is a logic-level type and can be switched safely using the Pi's 3.3V logic. If the threshold voltage were higher (like 4–5V), the Pi would not be able to drive the MOSFET properly.

Task 2: Building the Pull-Up Resistor

To complete Task 2, We built our own pull-up resistor circuit using a 10kΩ resistor and a push button. We connected one end of the button to ground and the other end to GPIO20 on the Raspberry Pi. The 10kΩ resistor was placed between GPIO20 and the 3.3V pin. This configuration ensures that when the button is not pressed, the input pin is pulled up to **HIGH (1)**. When the button is pressed, the pin is connected to ground and the value becomes **LOW (0)**.

We tested the circuit by printing the button value continuously in the terminal using pigpio. The output showed a steady "1" when the button was not pressed, and "0" when pressed, which confirmed that the pull-up resistor worked properly.

The images below show the button state in the Terminal.

Fig: Screenshot of task 2 output

Task 3: Running the Motor Using the Button

In Task 3, we connected the MOSFET gate to a GPIO pin so that pressing the button would turn the motor on. The motor ran smoothly when the button was pressed and stopped when released.

Code used:

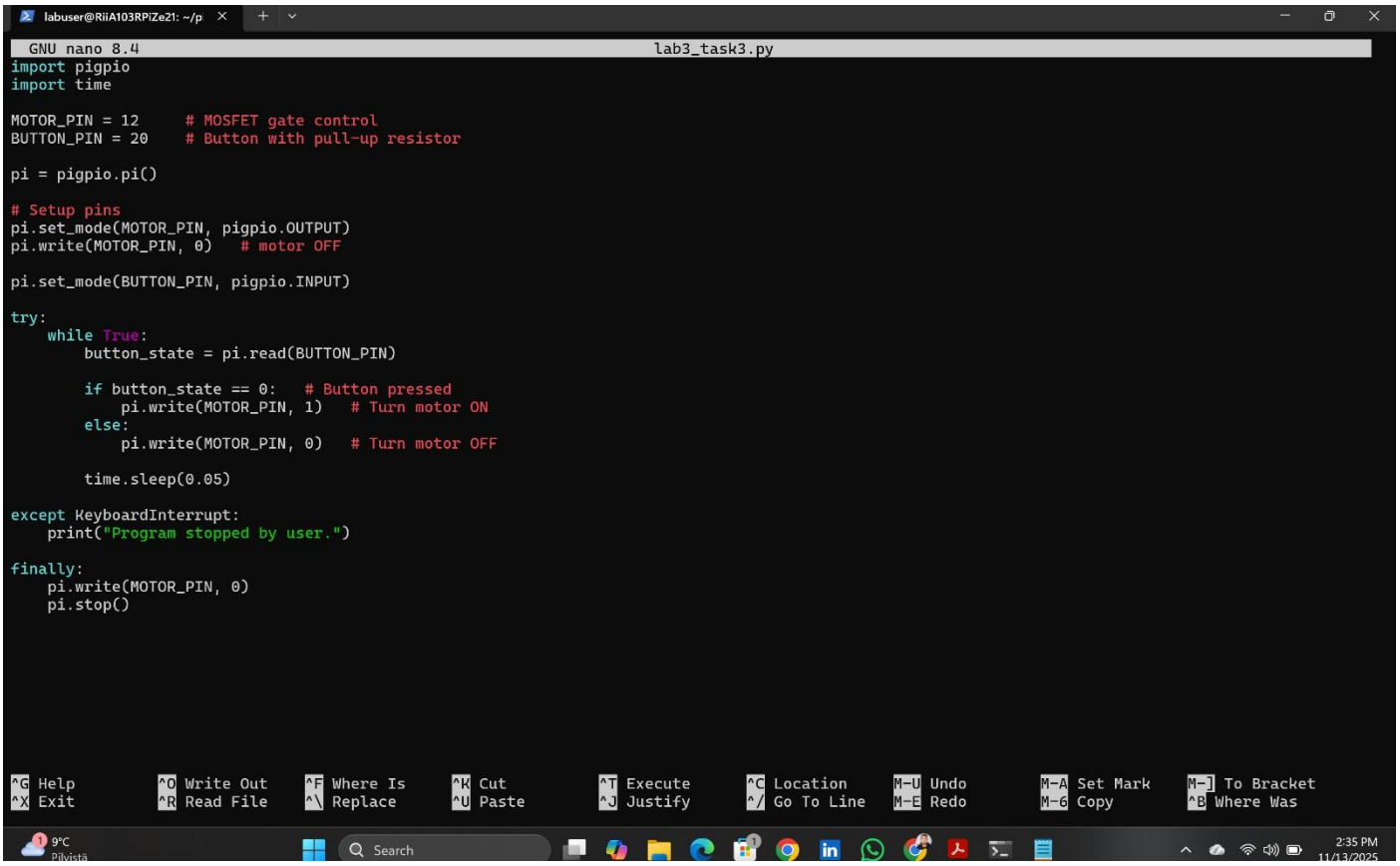

Fig: Screenshot of task 3 code

Explanation:

- When button_state == 0 → motor ON
- Else → motor OFF
- pi.write(MOTOR_PIN, 1) sets MOSFET gate HIGH → motor runs.

This confirmed that the MOSFET was switching correctly and that the Pi could control a 12V motor indirectly.

Task 4: Callback-Based Button Control

Task 4 required using a **callback** function from the pigpio library. A callback is triggered automatically when the input pin changes state, instead of constantly polling it in a loop. It is great because Callbacks allow event-driven programming and better debouncing.

Our callback code:

```
GNU nano 8.4                                    lab3_task4.py
import pigpio
import time

BUTTON_PIN = 20
MOTOR_PIN = 12

pi = pigpio.pi()

pi.set_mode(BUTTON_PIN, pigpio.INPUT)
pi.set_mode(MOTOR_PIN, pigpio.OUTPUT)
pi.write(MOTOR_PIN, 0)

# --- Callback function ---
def button_pressed(gpio, level, tick):
    print("Button pressed!")
    pi.write(MOTOR_PIN, 1)
    time.sleep(1)       # Motor runs for 1 second
    pi.write(MOTOR_PIN, 0)

# --- Register callback (falling edge = button press) ---
cb = pi.callback(BUTTON_PIN, pigpio.FALLING_EDGE, button_pressed)

try:
    print("Callback active. Press the button.")
    while True:
        time.sleep(1)   # Main loop does nothing

except KeyboardInterrupt:
    print("Program stopped.")

finally:
    cb.cancel()         # IMPORTANT: stop callback
    pi.write(MOTOR_PIN, 0)
    pi.stop()

                                        [ Read 34 lines ]
^G Help     ^O Write Out  ^F Where Is   ^K Cut     ^T Execute    ^C Location   M-U Undo     M-A Set Mark  M-] To Bracket
^X Exit     ^R Read File  ^\ Replace    ^U Paste   ^J Justify    ^/ Go To Line M-E Redo     M-6 Copy      ^B Where Was
```

Fig: Screenshot of task 4 code

And the output:

```
labuser@RiiA103RPiZe21:~/pigpio/milan_udit $ python3 lab3_task4.py
Callback active. Press the button.
Button pressed!
Button pressed!
Button pressed!
Button pressed!
Button pressed!
Button pressed!
Button pressed!
Button pressed!
Button pressed!
Button pressed!
Button pressed!
Button pressed!
Button pressed!
Button pressed!
^CProgram stopped.
labuser@RiiA103RPiZe21:~/pigpio/milan_udit $
```

Fig: Screenshot of task 4 output

Explanation:

- pi.callback(BUTTON_PIN, pigpio.FALLING_EDGE, button_pressed)
- Function is triggered only when the button transitions to LOW.
- This method reduces CPU usage and increases responsiveness.
- Debouncing is handled internally by pigpio.
- Callback canceled safely at end using cb.cancel().

Using callbacks made the button detection more responsive and cleaner. It also removed the need for while True loops constantly reading the pin.

4. Results and Analysis:

Task 1 Findings:

MOSFET successfully switches using Pi's 3.3V output thanks to low VGS(th). Threshold voltage verified from datasheet (1–2V) .

Task 2 Findings:

The pull-up resistor worked correctly; terminal showed stable "1" state until button pressed.

Task 3 Findings:

Motor runs reliably when button pressed. Correct MOSFET switching achieved. The flyback diode protected components—no resets or noise issues observed.

Task 4 Findings:

Callback method performed significantly better:

- No looping required
- Instant response on button press
- Clean console output

Everything behaved as expected, and We didn't face major issues once the wiring was correct. For the correction of our wiring and code for task 4 we got some help from our teacher **Aleksi Hayrynen.**

5. Conclusions:

This lab helped us to understand how to safely control higher-voltage components using the Raspberry Pi. We learned how MOSFETs actually behave in real circuits and why logic-level MOSFETs are needed for microcontroller projects. Building the pull-up resistor by hand made the concept much clearer compared to just enabling the internal pull-up.

The tasks also showed the importance of using a flyback diode when working with motors. Finally, working with callbacks gave us a better understanding of event-driven programming on the Pi.

Overall, the lab gave us practical experience with electronics, circuits, and GPIO programming, and the setup worked successfully for all tasks.

The End